LL(1) Grammar Analyzer

Assessment 10 — Results, Source Code, and Demo Screenshots

Course: Compiler Design Lab — BCSE307P

Team members:

- **S. Ajmal Afreed — 24BCE0754**
- **S. Likhil Kumar Reddy — 24BCE0452**
- **Karanam Kiran Naidu — 24BAI0202**

GitHub Repository

Public Repository Link: https://github.com/ajmalafreed/LL1-Grammar-Analyzer

Test Results :

Test Case	Input / Scenario	Observed Result	Status
FIRST set computation	Arithmetic expression grammar	FIRST sets calculated for all non-terminals	Passed
FOLLOW set computation	Arithmetic expression grammar	FOLLOW sets calculated for all non-terminals	Passed
Parsing table construction	Arithmetic expression grammar	LL(1) parsing table generated without conflicts	Passed
Valid input parsing	id + id * id	Input accepted by the predictive parser	Passed
Conflict detection	S -> a A | a B, A -> b, B -> c	Conflict detected at M[S, a]	Passed
Automatic left factoring	Grammar with common prefix a	Common prefix factored and transformed grammar re-analyzed	Passed
Grammar comparison	One LL(1) grammar and one conflicting grammar	Different grammar classifications displayed	Passed
Report export	Save analysis output	Report saved to LL1_Analysis_Report.txt	Passed

Source Code:

```
#include <iostream>
#include <string>
#include <vector>
#include <set>
#include <map>
#include <sstream>
#include <algorithm>
#include <iomanip>
#include <limits>
#include <fstream>
using namespace std;
// ============================================================
// CONSTANTS AND DATA STRUCTURES
// ============================================================
const string EPSILON = "epsilon";
const string END_MARKER = "$";
bool showSteps = false;
struct Production {
  string lhs;
  vector<string> rhs;
};
vector<Production> productions;
set<string> nonTerminals;
set<string> terminals;
map<string, set<string>> FIRST;
map<string, set<string>> FOLLOW;
map<string, map<string, vector<int>>> table;
// Captures console output while still displaying it, so the user can export
// the complete analysis transcript to a text report at the end.
class TeeBuffer : public streambuf {
private:
  streambuf* consoleBuffer;
  streambuf* logBuffer;
public:
  TeeBuffer(streambuf* console, streambuf* log)
    : consoleBuffer(console), logBuffer(log) {}
```

```
protected:
    int overflow(int ch) override {
        if (ch == traits_type::eof()) return traits_type::not_eof(ch);
        const int consoleResult = consoleBuffer->sputc(static_cast<char>(ch));
        const int logResult = logBuffer->sputc(static_cast<char>(ch));
        if (consoleResult == traits_type::eof() || logResult == traits_type::eof())
            return traits_type::eof();
        return ch;
    }
    int sync() override {
        const int a = consoleBuffer->pubsync();
        const int b = logBuffer->pubsync();
        return (a == 0 && b == 0) ? 0 : -1;
    }
};
// ============================================================
// UTILITY FUNCTIONS
// ============================================================
string trim(const string& text) {
    size_t start = text.find_first_not_of(" \t\r\n");
    if (start == string::npos) {
        return "";
    }
    size_t end = text.find_last_not_of(" \t\r\n");
    return text.substr(start, end - start + 1);
}
vector<string> tokenize(const string& text) {
    vector<string> tokens;
    string token;
    stringstream ss(text);
    while (ss >> token) {
        tokens.push_back(token);
    }
    return tokens;
}
string productionText(int index) {
    if (index < 0 ||
        index >= static_cast<int>(productions.size())) {
```

```
        return "";
    }
    const Production& p = productions[index];
    string result = p.lhs + " -> ";
    if (p.rhs.empty()) {
        result += EPSILON;
    } else {
        for (size_t i = 0; i < p.rhs.size(); i++) {
            if (i > 0) {
                result += " ";
            }
            result += p.rhs[i];
        }
    }
    return result;
}
bool isNonTerminal(const string& symbol) {
    return nonTerminals.count(symbol) > 0;
}
// ============================================================
// GRAMMAR INPUT
// ============================================================
bool readGrammar() {
    vector<string> inputLines;
    int choice;
    cout << "============================================================\n";
    cout << "              LL(1) GRAMMAR ANALYZER\n";
    cout << "============================================================\n";
    cout << "\nSELECT GRAMMAR\n";
    cout << "------------------------------------------------------------------------\n";
    cout << "1. Arithmetic Expression Grammar\n";
    cout << "2. Simple Assignment Grammar\n";
    cout << "3. Grammar with Epsilon Productions\n";
    cout << "4. Enter Custom Grammar\n";
    cout << "\nEnter your choice: ";
    if (!(cin >> choice) || choice < 1 || choice > 4) {
        cout << "\nError: Please enter a choice from 1 to 4.\n";
        return false;
```

```
}
cin.ignore(numeric_limits<streamsize>::max(), '\n');
if (choice == 1) {
    inputLines = {
        "E -> T E'",
        "E' -> + T E' | epsilon",
        "T -> F T'",
        "T' -> * F T' | epsilon",
        "F -> ( E ) | id"
    };
    cout << "\nLoaded Arithmetic Expression Grammar.\n";
} else if (choice == 2) {
    inputLines = {
        "S -> id = E",
        "E -> id + id"
    };
    cout << "\nLoaded Simple Assignment Grammar.\n";
} else if (choice == 3) {
    inputLines = {
        "S -> A B",
        "A -> a | epsilon",
        "B -> b | epsilon"
    };
    cout << "\nLoaded Grammar with Epsilon Productions.\n";
} else {
    int numberOfLines;
    cout << "\nGRAMMAR INPUT INSTRUCTIONS\n";
    cout << "--------------------------------------------------------------------------\n";
    cout << "1. Use -> between the left and right sides.\n";
    cout << "2. Separate symbols using spaces.\n";
    cout << "3. Use | for alternatives.\n";
    cout << "4. Use epsilon for an empty production.\n";
    cout << "5. Do not use $ as a grammar symbol.\n";
    cout << "\nEnter the number of production lines: ";
    if (!(cin >> numberOfLines) || numberOfLines <= 0) {
        cout << "\nError: Enter a positive number of production lines.\n";
        return false;
    }
```

```
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "\nEnter your grammar:\n";
    for (int i = 0; i < numberOfLines; i++) {
        string line;
        cout << "Production " << i + 1 << ": ";
        getline(cin, line);
        line = trim(line);
        if (line.empty()) {
            cout << "\nError: Production cannot be empty.\n";
            return false;
        }
        inputLines.push_back(line);
    }
}
// Parse the input productions.
for (const string& line : inputLines) {
    size_t arrowPosition = line.find("->");
    if (arrowPosition == string::npos) {
        cout << "\nError: Missing '->' in production:\n";
        cout << line << "\n";
        return false;
    }
    string lhs = trim(line.substr(0, arrowPosition));
    string rhsText = trim(line.substr(arrowPosition + 2));
    if (lhs.empty() || rhsText.empty()) {
        cout << "\nError: Invalid production:\n";
        cout << line << "\n";
        return false;
    }
    vector<string> lhsTokens = tokenize(lhs);
    if (lhsTokens.size() != 1 ||
        lhs == EPSILON ||
        lhs == END_MARKER ||
        lhs.find('|') != string::npos) {
        cout << "\nError: Invalid left-hand side: "
             << lhs << "\n";
        return false;
    }
```

```
    nonTerminals.insert(lhs);
    // Split alternatives separated by '|'.
    string alternative;
    stringstream rhsStream(rhsText);
    while (getline(rhsStream, alternative, '|')) {
      alternative = trim(alternative);
      if (alternative.empty()) {
        cout << "\nError: Empty alternative in production:\n";
        cout << line << "\n";
        return false;
      }
      vector<string> rhsTokens = tokenize(alternative);
      if (rhsTokens.empty()) {
        cout << "\nError: Empty alternative in production:\n";
        return false;
      }
       if (rhsTokens.size() > 1 &&
        find(rhsTokens.begin(), rhsTokens.end(), EPSILON)
          != rhsTokens.end()) {
        cout << "\nError: epsilon must appear alone "
          << "on the right-hand side.\n";
        return false;
      }
      if (find(rhsTokens.begin(), rhsTokens.end(), END_MARKER)
        != rhsTokens.end()) {
        cout << "\nError: '$' is reserved as the end marker.\n";
        return false;
      }
      productions.push_back({lhs,  rhsTokens});
    }
  }
  if (productions.empty()) {
    cout << "\nError: No productions were entered.\n";
    return false;
  }
  // Validate references that look like non-terminals. By convention,
  // this project uses uppercase names (such as E, Expr, E') for
  // non-terminals and lowercase/symbol names (such as id, +, a) for
```

```
    // terminals. This catches common spelling mistakes like E'' -> T X.
    for (const Production& p : productions) {
        for (const string& symbol : p.rhs) {
            if (symbol == EPSILON) {
                continue;
            }
            bool looksLikeNonTerminal =
                (!symbol.empty() && symbol[0] >= 'A' && symbol[0] <= 'Z');
            if (looksLikeNonTerminal && !nonTerminals.count(symbol)) {
                cout << "\nError: Undefined non-terminal '" << symbol
                    << "' used in production: " << productionText(
                        static_cast<int>(&p - productions.data()))
                    << "\nAdd a production for '" << symbol
                    << "' or correct the spelling.\n";
                return false;
            }
        }
    }
    // Identify terminals after checking likely non-terminal references.
    for (const Production& p : productions) {
        for (const string& symbol : p.rhs) {
            if (symbol != EPSILON &&
                !nonTerminals.count(symbol)) {
                terminals.insert(symbol);
            }
        }
    }
    // Initialize the FIRST and FOLLOW sets.
    for (const string& nt : nonTerminals) {
        FIRST[nt];
        FOLLOW[nt];
    }
    cout << "\nGrammar entered successfully.\n";
    return true;
}
// ============================================================
// DISPLAY GRAMMAR
// ============================================================
```

```
void displayGrammar() {
   cout << "\n============================================================\n";
   cout << "                ENTERED GRAMMAR\n";
   cout << "============================================================\n";
   for (size_t i = 0; i < productions.size(); i++) {
      cout << setw(3) << i + 1 << ". "
         << productionText(static_cast<int>(i)) << "\n";
   }
   cout << "\nStart symbol : " << productions[0].lhs << "\n";
   cout << "Non-terminals: ";
   for (const string& nt : nonTerminals) {
      cout << nt << " ";
   }
   cout << "\nTerminals   : ";
   for (const string& terminal : terminals) {
      cout << terminal << " ";
   }
   cout << "\n";
}
// ============================================================
// NEW FEATURE 1: DIRECT LEFT-RECURSION DETECTION
// ============================================================
void detectDirectLeftRecursion() {
   cout << "\n============================================================\n";
   cout << "           DIRECT LEFT-RECURSION CHECK\n";
   cout << "============================================================\n";
   bool found = false;
   for (const string& nt : nonTerminals) {
      for (const Production& p : productions) {
         if (p.lhs == nt && !p.rhs.empty() && p.rhs[0] == nt) {
            if (!found) {
               cout << "Direct left recursion detected:\n";
               found = true;
            }
            cout << "  " << productionText(static_cast<int>(&p - productions.data())) << "\n";
            cout << "  Explanation: " << nt << " can derive a string beginning with itself.\n";
            cout << "  Typical rewrite: " << nt << " -> beta " << nt << "' and "
               << nt << "' -> alpha " << nt << "' | epsilon (when applicable).\n";
```

```
        }
      }
    }
    if (!found) {
      cout << "No direct left recursion detected.\n";
    }
    cout << "Note: this check detects direct left recursion; it does not prove that indirect left recursion is absent.\n";
}
// ============================================================
// NEW FEATURE 2: CONSOLE GRAMMAR RELATIONSHIP VISUALIZATION
// ============================================================
void visualizeGrammar() {
    cout << "\n============================================================\n";
    cout << "          GRAMMAR RELATIONSHIP VIEW\n";
    cout << "============================================================\n";
    cout << "Start symbol: " << (productions.empty() ? "(none)" : productions[0].lhs) << "\n\n";
    cout << "Production paths (non-terminals point to symbols on their RHS):\n";
    for (const string& nt : nonTerminals) {
      cout << "[" << nt << "]\n";
      bool any = false;
      for (const Production& prod : productions) {
        if (prod.lhs != nt) continue;
        any = true;
        cout << "  +-- ";
        if (prod.rhs.empty() || (prod.rhs.size() == 1 && prod.rhs[0] == EPSILON)) {
          cout << "epsilon (empty string)";
        } else {
          for (size_t i = 0; i < prod.rhs.size(); ++i) {
            if (i) cout << " ";
            const string& sym = prod.rhs[i];
            if (nonTerminals.count(sym)) cout << "[" << sym << "]";
            else if (sym == EPSILON) cout << "epsilon";
            else cout << "'" << sym << "'";
          }
        }
        cout << "\n";
      }
      if (!any) cout << "  +-- (no productions)\n";
```

```
    }
    cout << "\nLegend: [Name] = non-terminal; 'token' = terminal.\n";
}
// ============================================================
// FIRST OF A SEQUENCE
// ============================================================
set<string> firstOfSequence(const vector<string>& sequence) {
    set<string> result;
    if (sequence.empty()) {
        result.insert(EPSILON);
        return result;
    }
    bool allNullable = true;
    for (const string& symbol : sequence) {
        if (symbol == EPSILON) {
            result.insert(EPSILON);
            break;
        }
        if (terminals.count(symbol)) {
            result.insert(symbol);
            allNullable = false;
            break;
        }
        bool containsEpsilon = false;
        for (const string& item : FIRST[symbol]) {
            if (item == EPSILON) {
                containsEpsilon = true;
            } else {
                result.insert(item);
            }
        }
        if (!containsEpsilon) {
            allNullable = false;
            break;
        }
    }
    if (allNullable) {
        result.insert(EPSILON);
```

```
    }
    return result;
}
// ============================================================
// COMPUTE FIRST SETS
// ============================================================
void computeFIRST() {
    bool changed;
    cout << "\n================================================================\n";
    cout << "                FIRST SET CALCULATION\n";
    cout << "================================================================\n";
    do {
        changed = false;
        for (size_t index = 0; index < productions.size(); index++) {
            const Production& p = productions[index];
            string A = p.lhs;
            if (showSteps) {
                cout << "\nProduction: "
                    << productionText(static_cast<int>(index))
                    << "\n";
            }
            bool allNullable = true;
            for (const string& symbol : p.rhs) {
                if (symbol == EPSILON) {
                    if (FIRST[A].insert(EPSILON).second) {
                        changed = true;
                        if (showSteps) {
                            cout << "  Added epsilon to FIRST("
                                << A << ") because the production "
                                << "derives epsilon.\n";
                        }
                    }
                    break;
                }
                if (terminals.count(symbol)) {
                    if (FIRST[A].insert(symbol).second) {
                        changed = true;
                        if (showSteps) {
```

```
                cout << "  Added '" << symbol
                   << "' to FIRST(" << A
                    << ") because it is the first "
                   << "terminal in this production.\n";
            }
        }
        allNullable = false;
        break;
    }
    bool containsEpsilon = false;
    for (const string& item : FIRST[symbol]) {
        if (item == EPSILON) {
            containsEpsilon = true;
        } else if (FIRST[A].insert(item).second) {
            changed = true;
            if (showSteps) {
                cout << "  Added '" << item
                   << "' to FIRST(" << A
                   << ") from FIRST("
                   << symbol << ").\n";
            }
        }
    }
    if (!containsEpsilon) {
        allNullable  =  false;
        break;
    }
    if (showSteps) {
        cout << "  " << symbol
            << " can derive epsilon; examining "
            << "the next symbol.\n";
    }
}
if (allNullable) {
    if (FIRST[A].insert(EPSILON).second) {
        changed = true;
        if (showSteps) {
            cout << "  Added epsilon to FIRST("
```

```
                    << A << ") because every symbol "
                    << "on the RHS can derive epsilon.\n";
                }
            }
        }
    }
  } while (changed);
  cout << "\nFINAL FIRST SETS\n";
  cout << "-----------------------------------------------------------------------\n";
  for (const string& nt : nonTerminals) {
    cout << left << setw(12) << ("FIRST(" + nt + ")");
    cout << "= { ";
    bool first = true;
    for (const string& symbol : FIRST[nt]) {
      if (!first) {
        cout << ", ";
      }
      cout << symbol;
      first = false;
    }
    cout << " }\n";
  }
}
// ============================================================
// COMPUTE FOLLOW SETS
// ============================================================
void computeFOLLOW() {
  bool changed;
  cout << "\n============================================================\n";
  cout << "              FOLLOW SET CALCULATION\n";
  cout << "============================================================\n";
  string startSymbol = productions[0].lhs;

  if (FOLLOW[startSymbol].insert(END_MARKER).second) {
    if (showSteps) {
      cout << "Added '$' to FOLLOW("
        << startSymbol
        << ") because it is the start symbol.\n";
```

```
    }
  }
  do {
    changed = false;
    for (const Production& p : productions) {
      string A = p.lhs;
      for (size_t i = 0; i < p.rhs.size(); i++) {
        string B = p.rhs[i];
        if (!nonTerminals.count(B)) {
          continue;
        }
        vector<string> beta;
        for (size_t j = i + 1; j < p.rhs.size(); j++) {
          beta.push_back(p.rhs[j]);
        }
        if (showSteps) {
          cout << "\nProduction: "
            << productionText(
              static_cast<int>(&p - productions.data())
            )
            << "\nExamining non-terminal '"
            << B << "'.\n";
        }
        set<string> firstBeta = firstOfSequence(beta);
        // FIRST(beta) minus epsilon goes into FOLLOW(B).
        for (const string& symbol : firstBeta) {
          if (symbol != EPSILON &&
            FOLLOW[B].insert(symbol).second) {
            changed = true;
            if (showSteps) {
              cout << "  Added '" << symbol
                << "' to FOLLOW(" << B
                << ") from FIRST of the "
                << "sequence following it.\n";
            }
          }
        }
        // If beta is nullable, FOLLOW(A) goes into FOLLOW(B).
```

```
            bool betaNullable =
                beta.empty() ||
                firstBeta.count(EPSILON) > 0;
            if (betaNullable) {
                if (showSteps) {
                    cout << "  The sequence following "
                        << B << " is empty or nullable.\n";
                    cout << "  Propagating FOLLOW("
                        << A << ") to FOLLOW("
                        << B << ").\n";
                }
                for (const string& symbol : FOLLOW[A]) {
                    if (FOLLOW[B].insert(symbol).second) {
                        changed = true;
                        if (showSteps) {
                            cout << "  Added '" << symbol
                                << "' to FOLLOW(" << B
                                << ") from FOLLOW("
                                << A << ").\n";
                        }
                    }
                }
            }
        }
    } while (changed);
    cout << "\nFINAL FOLLOW SETS\n";
    cout << "-------------------------------------------------------------------------\n";
    for (const string& nt : nonTerminals) {
        cout << left << setw(12) << ("FOLLOW(" + nt + ")");
        cout << "= { ";
        bool first = true;
        for (const string& symbol : FOLLOW[nt]) {
            if (!first) {
                cout << ", ";
            }
            cout << symbol;
            first = false;
```

```
        }
        cout << " }\n";
    }
}
// ============================================================
// BUILD PARSING TABLE
// ============================================================
void buildParsingTable() {
    cout << "\nParsing table generated successfully.\n";
    for (size_t i = 0; i < productions.size(); i++) {
        const Production& p = productions[i];
        set<string> firstSet = firstOfSequence(p.rhs);
        for (const string& symbol : firstSet) {
            if (symbol != EPSILON) {
                table[p.lhs][symbol].push_back(
                    static_cast<int>(i)
                );
            }
        }
        if (firstSet.count(EPSILON)) {
            for (const string& symbol : FOLLOW[p.lhs]) {
                table[p.lhs][symbol].push_back(
                    static_cast<int>(i)
                );
            }
        }
    }
}
// ============================================================
// DISPLAY A COMPACT PARSING TABLE
// ============================================================
void printParsingTable() {
    cout << "\n";
    cout << "============================================================\n";
    cout << "                LL(1) PARSING TABLE\n";
    cout << "============================================================\n\n";
    set<string> columns = terminals;
    columns.insert(END_MARKER);
```

```
    const int rowWidth = 15;
    const int cellWidth = 22;
    cout << left << setw(rowWidth) << "NON-TERMINAL";
    for (const string& terminal : columns) {
        cout << left << setw(cellWidth) << terminal;
    }
    cout << "\n";
    cout << string(
        rowWidth + cellWidth * static_cast<int>(columns.size()),
        '-'
    ) << "\n";
    for (const string& nt : nonTerminals) {
        cout << left << setw(rowWidth) << nt;
        for (const string& terminal : columns) {
            string cell = "-";
            auto row = table.find(nt);
            if (row != table.end()) {
                auto entry = row->second.find(terminal);
                if (entry != row->second.end()) {
                    if (entry->second.size() > 1) {
                        cell = "CONFLICT";
                    } else {
                        cell = productionText(entry->second[0]);
                    }
                }
            }
            if (cell.length() > static_cast<size_t>(cellWidth - 2)) {
                cell = cell.substr(0, cellWidth - 5) + "...";
            }
            cout << left << setw(cellWidth) << cell;
        }
        cout << "\n";
    }
    cout << "\nLegend:\n";
    cout << "  -        : Empty table cell\n";
    cout << "  CONFLICT : Multiple productions in the same cell\n";
    cout << "  A -> ...  : Production selected by the parser\n";
}
```

```
// ============================================================
// CONFLICT DETECTION
// ============================================================
bool detectConflicts() {
   bool conflict = false;
   cout << "\n============================================================\n";
   cout << "                CONFLICT ANALYSIS\n";
   cout << "============================================================\n";
   for (const string& nt : nonTerminals) {
      for (const auto& entry : table[nt]) {
         if (entry.second.size() > 1) {
            conflict = true;
            cout << "\nConflict at M["
                 << nt << ", " << entry.first << "]\n";
            cout << "Conflicting productions:\n";
            int count = 1;
            for (int index : entry.second) {
               cout << "  " << count++ << ". "
                    << productionText(index) << "\n";
            }
            cout << "\nExplanation:\n";
            cout << "The lookahead symbol '" << entry.first
                 << "' selects more than one production for "
                 << nt << ".\n";
            cout << "A predictive parser therefore cannot choose "
                 << "one production uniquely.\n";
            // Check whether the conflicting alternatives share
            // their first RHS symbol, which may indicate a
            // left-factoring opportunity.
            bool sharedFirstSymbol = true;
            string commonFirst;
            bool firstProduction = true;
            for (int index : entry.second) {
               const vector<string>& rhs = productions[index].rhs;
               string firstSymbol = rhs.empty() ? EPSILON : rhs[0];
               if (firstProduction) {
                  commonFirst = firstSymbol;
                  firstProduction = false;
```

```
            } else if (firstSymbol != commonFirst) {
                sharedFirstSymbol = false;
            }
        }
        if (sharedFirstSymbol && commonFirst != EPSILON) {
            cout << "\nPossible improvement:\n";
            cout << "The conflicting alternatives begin with '"
                 << commonFirst << "'. Consider left factoring "
                 << "the grammar, then analyze the transformed "
                 << "grammar again.\n";
        } else {
            cout << "\nPossible improvement:\n";
            cout << "Review the FIRST and FOLLOW sets for these "
                 << "productions. Depending on the cause, grammar "
                 << "rewriting (such as left factoring or removing "
                 << "left recursion) may help. Recheck the language "
                 << "after any change.\n";
        }
        cout << "Conclusion: The grammar is NOT LL(1).\n";
        cout << "------------------------------------------------------------------\n";
      }
    }
  }
  if (!conflict) {
    cout << "No parsing-table conflicts detected.\n";
    cout << "Every occupied cell contains one production.\n";
  }
  return conflict;
}
// ============================================================
// PREDICTIVE PARSER
// ============================================================
void predictiveParser() {
  cout << "\n";
  cout << "============================================================\n";
  cout << "                PARSING TRACE\n";
  cout << "============================================================\n\n";
  cout << "Enter tokens separated by spaces.\n";
```

```
cout << "Example: id + id * id\n";
cout << "For empty input, enter epsilon.\n";
cout << "\nInput string: ";
string inputLine;
getline(cin >> ws, inputLine);
vector<string> input = tokenize(inputLine);
if (input.size() == 1 && input[0] == EPSILON) {
   input.clear();
}
if (find(input.begin(), input.end(), EPSILON) != input.end()) {
   cout << "\nREJECTED: epsilon must be entered alone.\n";
   return;
}
for (size_t i = 0; i < input.size(); ++i) {
   const string& token = input[i];
   if (!terminals.count(token)) {
      cout << "\nPARSING ERROR\n";
      cout << "Unexpected token: '" << token << "'\n";
      cout << "Token position: " << (i + 1) << "\n";
      cout << "Reason: this token does not appear among the grammar's terminals.\n";
      cout << "Action: check the token spelling or revise the grammar.\n";
      cout << "Result: REJECTED\n";
      return;
   }
}
input.push_back(END_MARKER);
vector<string> parserStack;
parserStack.push_back(END_MARKER);
parserStack.push_back(productions[0].lhs);
size_t inputPosition = 0;
bool accepted = false;
int stepCount = 0;
const int MAX_STEPS = 10000;
const int stackWidth = 26;
const int inputWidth = 26;
cout << left
   << setw(stackWidth) << "STACK"
   << setw(inputWidth) << "REMAINING INPUT"
```

```
    << "ACTION\n";
cout << string(85, '-') << "\n";
while (!parserStack.empty() &&
       inputPosition < input.size() &&
       stepCount < MAX_STEPS) {
    stepCount++;
    string stackText;
    string inputText;
    for (const string& symbol : parserStack) {
        stackText += symbol + " ";
    }
    for (size_t i = inputPosition; i < input.size(); i++) {
        inputText += input[i] + " ";
    }
    cout << left
         << setw(stackWidth) << stackText
         << setw(inputWidth) << inputText;
    string top = parserStack.back();
    string lookahead = input[inputPosition];
    if (top == END_MARKER && lookahead == END_MARKER) {
        cout << "ACCEPT\n";
        accepted = true;
        break;
    }
    if (!isNonTerminal(top) && top != END_MARKER) {
        if (top == lookahead) {
            cout << "Match " << top << "\n";
            parserStack.pop_back();
            inputPosition++;
        } else {
            cout << "PARSING ERROR: terminal mismatch\n";
            cout << "  Input position: " << (inputPosition + 1) << "\n";
            cout << "  Expected token: '" << top << "'\n";
            cout << "  Actual token:   '" << lookahead << "'\n";
            cout << "  Reason: the next input token does not match the terminal on top of the parser stack.\n";
            cout << "  Action: check for a missing, extra, or misspelled token near this position.\n";
            break;
        }
```

```
    continue;
}
auto row = table.find(top);
if (row == table.end()) {
    cout << "PARSING ERROR: no parsing-table row for non-terminal '" << top << "'.\n";
    cout << "  Input position: " << (inputPosition + 1) << "; lookahead: '" << lookahead << "'.\n";
    cout << "  Reason: the current stack symbol is not available in the parsing table.\n";
    break;
}
auto cell = row->second.find(lookahead);
if (cell == row->second.end()) {
    cout << "PARSING ERROR: empty parsing-table cell M[" << top
        << ", " << lookahead << "]\n";
    cout << "  Input position: " << (inputPosition + 1) << "\n";
    cout << "  Reason: no production allows non-terminal '" << top
        << "' to proceed when the lookahead is '" << lookahead << "'.\n";
    cout << "  Action: check for a missing/extra token or revise the grammar.\n";
    break;
}
if (cell->second.size() != 1) {
    cout << "PARSING ERROR: parsing-table conflict at M["
        << top << ", " << lookahead << "]\n";
    cout << "  Reason: more than one production matches this lookahead, so parsing is ambiguous for this table.\n";
    cout << "  Action: remove the grammar conflict before predictive parsing.\n";
    break;
}
int productionIndex = cell->second[0];
cout << productionText(productionIndex) << "\n";
parserStack.pop_back();
const vector<string>& rhs = productions[productionIndex].rhs;
// Do not push epsilon onto the stack.
if (!(rhs.size() == 1 && rhs[0] == EPSILON)) {
    for (int i = static_cast<int>(rhs.size()) - 1;
        i >= 0;
        i--) {
       parserStack.push_back(rhs[i]);
    }
}
```

```
    }
    if (stepCount >= MAX_STEPS) {
        cout << "\nParser stopped: maximum step count reached.\n";
        accepted = false;
    }
    cout << "\n============================================================\n";
    cout << "                PARSING RESULT\n";
    cout << "============================================================\n";
    if (accepted) {
        cout << "ACCEPTED: The input string belongs to the grammar.\n";
    } else {
        cout << "REJECTED: The input string could not be parsed.\n";
    }
}
// ============================================================
// GRAMMAR COMPARISON DEMONSTRATION
// ============================================================
void resetGrammarState() {
    productions.clear();
    nonTerminals.clear();
    terminals.clear();
    FIRST.clear();
    FOLLOW.clear();
    table.clear();
}
bool loadGrammarLines(const vector<string>& lines) {
    for (const string& line : lines) {
        size_t arrowPosition = line.find("->");
        if (arrowPosition == string::npos) return false;
        string lhs = trim(line.substr(0, arrowPosition));
        string rhsText = trim(line.substr(arrowPosition + 2));
        if (lhs.empty() || rhsText.empty()) return false;
        nonTerminals.insert(lhs);
        string alternative;
        stringstream rhsStream(rhsText);
        while (getline(rhsStream, alternative, '|')) {
            alternative = trim(alternative);
            if (alternative.empty()) return false;
```

```
            vector<string> rhs = tokenize(alternative);
            if (rhs.size() == 1 && rhs[0] == EPSILON) rhs.clear();
            else {
              for (const string& symbol : rhs) {
                if (symbol == EPSILON || symbol == END_MARKER) return false;
              }
            }
            productions.push_back({lhs, rhs});
          }
        }
        for (const Production& p : productions) {
          for (const string& symbol : p.rhs) {
            if (!isNonTerminal(symbol)) terminals.insert(symbol);
          }
        }
        return !productions.empty();
    }
    bool analyzeComparisonGrammar(const string& label, const vector<string>& lines) {
        resetGrammarState();
        cout << "\n============================================================\n";
        cout << "              " << label << "\n";
        cout << "============================================================\n";
        if (!loadGrammarLines(lines)) {
          cout << "Could not load comparison grammar.\n";
          return false;
        }
        displayGrammar();
        computeFIRST();
        computeFOLLOW();
        buildParsingTable();
        printParsingTable();
        bool conflict = detectConflicts();
        cout << "Classification: " << (conflict ? "NOT LL(1)" : "LL(1)") << "\n";
        return !conflict;
    }
    vector<string> readGrammarLinesFromKeyboard(const string& label) {
        vector<string> lines;
        int count;
```

```
    cout << "\nEnter " << label << " production-line count: ";
    if (!(cin >> count) || count <= 0 || count > 100) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Error: enter a number from 1 to 100.\n";
        return {};
    }
    cin.ignore(numeric_limits<streamsize>::max(),  '\n');
    cout << "Enter each production on a separate line.\n";
    cout << "Use ->, spaces between symbols, | for alternatives, and epsilon for empty.\n";
    for (int i = 0; i < count; ++i) {
        string line;
        cout << label << " production " << (i + 1) << ": ";
        getline(cin, line);
        line = trim(line);
        if (line.empty()) {
            cout << "Error: production cannot be empty.\n";
            return {};
        }
        lines.push_back(line);
    }
    return lines;
}
void compareUserGrammars() {
    cout << "\nGRAMMAR COMPARISON\n";
    cout << "Enter the two grammars separately.\n";
    vector<string> grammarA = readGrammarLinesFromKeyboard("Grammar A");
    if (grammarA.empty()) return;
    vector<string> grammarB = readGrammarLinesFromKeyboard("Grammar B");
    if (grammarB.empty()) return;

    bool resultA = analyzeComparisonGrammar("GRAMMAR A", grammarA);
    bool resultB = analyzeComparisonGrammar("GRAMMAR B", grammarB);
    cout << "\n============================================================\n";
    cout << "              COMPARISON SUMMARY\n";
    cout << "============================================================\n";
    cout << "Grammar A: " << (resultA ? "LL(1)" : "NOT LL(1) or invalid") << "\n";
    cout << "Grammar B: " << (resultB ? "LL(1)" : "NOT LL(1) or invalid") << "\n";
```

```
    if (resultA && resultB) {
        cout << "Both grammars are LL(1).\n";
    } else if (resultA && !resultB) {
        cout << "Grammar A is LL(1), while Grammar B has a conflict or could not be analyzed.\n";
    } else if (!resultA && resultB) {
        cout << "Grammar B is LL(1), while Grammar A has a conflict or could not be analyzed.\n";
    } else {
        cout << "Neither grammar was classified as LL(1); inspect each analysis above.\n";
    }
}
// ============================================================
// AUTOMATIC LEFT FACTORING
// ============================================================
vector<string> normalizedRHS(const vector<string>& rhs) {
    if (rhs.size() == 1 && rhs[0] == EPSILON) return {};
    return rhs;
}
string uniqueFactoredName(const string& lhs, const set<string>& usedNames) {
    string candidate = lhs + "'";
    while (usedNames.count(candidate)) candidate += "'";
    return candidate;
}
vector<Production> leftFactorGrammar(const vector<Production>& input, bool& changed) {
    vector<Production> current = input;
    changed = false;
    // Repeatedly factor the longest common prefix shared by any two
    // alternatives belonging to the same non-terminal.
    for (int pass = 0; pass < 100; ++pass) {
        int bestLength = 0;
        string bestLhs;
        vector<string> bestPrefix;
        vector<size_t> bestIndices;
        set<string> usedNames;
        for (const Production& p : current) usedNames.insert(p.lhs);
        for (size_t i = 0; i < current.size(); ++i) {
            vector<string> left = normalizedRHS(current[i].rhs);
            for (size_t j = i + 1; j < current.size(); ++j) {
                if (current[i].lhs != current[j].lhs) continue;
```

```
      vector<string> right = normalizedRHS(current[j].rhs);
      size_t common = 0;
      while (common < left.size() && common < right.size() &&
             left[common] == right[common]) {
        ++common;
      }
      if (common > static_cast<size_t>(bestLength)) {
        bestLength = static_cast<int>(common);
        bestLhs = current[i].lhs;
        bestPrefix.assign(left.begin(), left.begin() + common);
      }
    }
  }
  if (bestLength == 0) break;
  // Gather every alternative that starts with the selected prefix.
  for (size_t i = 0; i < current.size(); ++i) {
    if (current[i].lhs != bestLhs) continue;
    vector<string> rhs = normalizedRHS(current[i].rhs);
    if (rhs.size() < bestPrefix.size()) continue;
    if (equal(bestPrefix.begin(), bestPrefix.end(), rhs.begin()))
      bestIndices.push_back(i);
  }
  if (bestIndices.size() < 2) break;
  string newName = uniqueFactoredName(bestLhs, usedNames);
  vector<Production> next;
  size_t firstIndex = bestIndices.front();
  set<size_t> factoredIndices(bestIndices.begin(), bestIndices.end());
  for (size_t i = 0; i < current.size(); ++i) {
    if (i == firstIndex) {
      vector<string> parentRhs = bestPrefix;
      parentRhs.push_back(newName);
      next.push_back({bestLhs, parentRhs});
    }
    if (!factoredIndices.count(i)) next.push_back(current[i]);
  }
  // Add the suffix alternatives under the generated non-terminal.
  for (size_t index : bestIndices) {
    vector<string> rhs = normalizedRHS(current[index].rhs);
```

```
            vector<string> suffix(rhs.begin() + bestPrefix.size(), rhs.end());
            if (suffix.empty()) suffix.push_back(EPSILON);
            next.push_back({newName,  suffix});
        }
        current.swap(next);
        changed = true;
    }
    return current;
}
void rebuildGrammarSymbols() {
    nonTerminals.clear();
    terminals.clear();
    FIRST.clear();
    FOLLOW.clear();
    table.clear();
    for (const Production& p : productions) nonTerminals.insert(p.lhs);
    for (const Production& p : productions) {
        for (const string& symbol : p.rhs) {
            if (symbol != EPSILON && !nonTerminals.count(symbol))
                terminals.insert(symbol);
        }
    }
    for (const string& nt : nonTerminals) {
        FIRST[nt];
        FOLLOW[nt];
    }
}
void printGrammarAsProductions(const vector<Production>& grammar) {
    for (const Production& p : grammar) {
        cout << p.lhs << " -> ";
        if (p.rhs.empty() || (p.rhs.size() == 1 && p.rhs[0] == EPSILON)) {
            cout << EPSILON;
        } else {
            for (size_t i = 0; i < p.rhs.size(); ++i) {
                if (i) cout << ' ';
                cout << p.rhs[i];
            }
        }
```

```
        cout << "\n";
    }
}
void offerLeftFactoring() {
    char choice;
    cout << "\nWould you like to generate and analyze a left-factored version? (y/n): ";
    if (!(cin >> choice) || (choice != 'y' && choice != 'Y')) return;
    bool changed = false;
    vector<Production> factored = leftFactorGrammar(productions, changed);
    if (!changed) {
        cout << "No shared non-empty prefixes were found, so automatic left factoring "
             << "could not transform this grammar.\n";
        return;
    }
    cout << "\n============================================================\n";
    cout << "          AUTOMATIC LEFT-FACTORING RESULT\n";
    cout << "============================================================\n";
    cout << "Generated grammar (review it before using it in a formal submission):\n";
    printGrammarAsProductions(factored);
    cout << "\nThe analyzer will now recompute FIRST, FOLLOW, and the parsing table.\n";
    productions = factored;
    rebuildGrammarSymbols();
    displayGrammar();
    computeFIRST();
    computeFOLLOW();
    buildParsingTable();
    printParsingTable();
    bool remainingConflicts = detectConflicts();
    cout << "\nClassification after left factoring: "
         << (remainingConflicts ? "NOT LL(1)" : "LL(1)") << "\n";
    if (remainingConflicts) {
        cout << "Left factoring alone did not remove every conflict. Review the new diagnostics.\n";
    } else {
        char parseChoice;
        cout << "\nDo you want to test an input string with the transformed grammar? (y/n): ";
        if (cin >> parseChoice && (parseChoice == 'y' || parseChoice == 'Y'))
            predictiveParser();
    }
```

```
}
// ============================================================
// MAIN
// ============================================================
int runProgram() {
    cout << "============================================================\n";
    cout << "              LL(1) GRAMMAR ANALYZER\n";
    cout << "============================================================\n";
    cout << "1. Analyze one grammar\n";
    cout << "2. Compare two grammars entered from keyboard\n";
    cout << "Enter your choice: ";
    int mainChoice;
    if (!(cin >> mainChoice) || (mainChoice != 1 && mainChoice != 2)) {
        cout << "Invalid choice. Please enter 1 or 2.\n";
        return 1;
    }
    if (mainChoice == 2) {
        compareUserGrammars();
        return 0;
    }
    if (!readGrammar()) {
        cout << "\nProgram terminated due to invalid grammar input.\n";
        return 1;
    }
    displayGrammar();
    detectDirectLeftRecursion();
    visualizeGrammar();
    char choice;
    cout << "\nShow step-by-step FIRST and FOLLOW calculations? (y/n): ";
    if (!(cin >> choice)) {
        cout << "\nInvalid input.\n";
        return 1;
    }
    showSteps = (choice == 'y' || choice == 'Y');
    computeFIRST();
    computeFOLLOW();
    buildParsingTable();
    printParsingTable();
```

```
    bool hasConflicts = detectConflicts();
    cout << "\n============================================================\n";
    cout << "              GRAMMAR CLASSIFICATION\n";
    cout << "============================================================\n";
    if (hasConflicts) {
        cout << "Grammar is NOT LL(1).\n";
        cout << "Predictive parsing is skipped because the table has conflicts.\n";
        offerLeftFactoring();
        return 0;
    }
    cout << "Grammar is LL(1).\n";
    char parseChoice;
    cout << "\nDo you want to test an input string? (y/n): ";
    if (!(cin >> parseChoice)) {
        return 0;
    }
    if (parseChoice == 'y' || parseChoice == 'Y') {
        predictiveParser();
    }
    cout << "\n============================================================\n";
    cout << "            LL(1) ANALYSIS COMPLETED\n";
    cout << "============================================================\n";
    return 0;
}
int main() {
    ostringstream report;
    streambuf* originalBuffer = cout.rdbuf();
    TeeBuffer tee(originalBuffer, report.rdbuf());
    cout.rdbuf(&tee);
    int status = runProgram();
    cout.flush();
    cout.rdbuf(originalBuffer);
    cout << "\nSave the console analysis to LL1_Analysis_Report.txt? (y/n): ";
    char saveChoice;
    if (cin >> saveChoice && (saveChoice == 'y' || saveChoice == 'Y')) {
        ofstream output("LL1_Analysis_Report.txt");
        if (!output) {
            cout << "Could not create LL1_Analysis_Report.txt in the current folder.\n";
```

```
        return status == 0 ? 1 : status;
    }
    output << report.str();
    output.close();
    cout << "Report saved as LL1_Analysis_Report.txt in the current working directory.\n";
  } else {
    cout << "Report not saved.\n";
  }
  return status;
}
```

DEMO SCREENSHOTS :

```
PS C:\Users\1ajma\OneDrive\ドキュメント\LL1-Grammar-Analyzer> g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o main.exe
PS C:\Users\1ajma\OneDrive\ドキュメント\LL1-Grammar-Analyzer> .\main.exe
============================================================
                  LL(1) GRAMMAR ANALYZER
============================================================
1. Analyze one grammar
2. Compare two grammars entered from keyboard
Enter your choice: 1
============================================================
                  LL(1) GRAMMAR ANALYZER
============================================================

SELECT GRAMMAR
------------------------------------------------------------
1. Arithmetic Expression Grammar
2. Simple Assignment Grammar
3. Grammar with Epsilon Productions
4. Enter Custom Grammar

Enter your choice: 1

Loaded Arithmetic Expression Grammar.

Grammar entered successfully.

============================================================
                    ENTERED GRAMMAR
============================================================
  1. E -> T E'
  2. E' -> + T E'
  3. E' -> epsilon
  4. T -> F T'
  5. T' -> * F T'
  6. T' -> epsilon
  7. F -> ( E )
  8. F -> id
```

```
Start symbol : E
Non-terminals: E E' F T T'
Terminals    : ( ) * + id

============================================================
              DIRECT LEFT-RECURSION CHECK
============================================================
No direct left recursion detected.
Note: this check detects direct left recursion; it does not prove that indirect left recursion is absent.

============================================================
            GRAMMAR RELATIONSHIP VIEW
============================================================
Start symbol: E

Production paths (non-terminals point to symbols on their RHS):
[E]
  +-- [T] [E']
[E']
  +-- '+' [T] [E']
  +-- epsilon (empty string)
[F]
  +-- '(' [E] ')'
  +-- 'id'
[T]
  +-- [F] [T']
[T']
  +-- '*' [F] [T']
  +-- epsilon (empty string)

Legend: [Name] = non-terminal; 'token' = terminal.

Show step-by-step FIRST and FOLLOW calculations? (y/n): y
```

```
============================================================
                  FIRST SET CALCULATION
============================================================

Production: E -> T E'

Production: E' -> + T E'
  Added '+' to FIRST(E') because it is the first terminal in this production.

Production: E' -> epsilon
  Added epsilon to FIRST(E') because the production derives epsilon.

Production: T -> F T'

Production: T' -> * F T'
  Added '*' to FIRST(T') because it is the first terminal in this production.

Production: T' -> epsilon
  Added epsilon to FIRST(T') because the production derives epsilon.

Production: F -> ( E )
  Added '(' to FIRST(F) because it is the first terminal in this production.

Production: F -> id
  Added 'id' to FIRST(F) because it is the first terminal in this production.

Production: E -> T E'

Production: E' -> + T E'

Production: E' -> epsilon

Production: T -> F T'
  Added '(' to FIRST(T) from FIRST(F).
  Added 'id' to FIRST(T) from FIRST(F).

Production: T' -> * F T'

Production: T' -> epsilon

Production: F -> ( E )

Production: F -> id
```

```
Production: E -> T E'
  Added '(' to FIRST(E) from FIRST(T).
  Added 'id' to FIRST(E) from FIRST(T).

Production: E' -> + T E'

Production: E' -> epsilon

Production: T -> F T'

Production: T' -> * F T'

Production: T' -> epsilon

Production: F -> ( E )

Production: F -> id

Production: E -> T E'

Production: E' -> + T E'

Production: E' -> epsilon

Production: T -> F T'

Production: T' -> * F T'

Production: T' -> epsilon

Production: F -> ( E )

Production: F -> id

FINAL FIRST SETS
------------------------------------------------------------
FIRST(E)    = { (, id }
FIRST(E')   = { +, epsilon }
FIRST(F)    = { (, id }
FIRST(T)    = { (, id }
FIRST(T')   = { *, epsilon }
```

```
============================================================
                    FOLLOW SET CALCULATION
============================================================
Added '$' to FOLLOW(E) because it is the start symbol.

Production: E -> T E'
Examining non-terminal 'T'.
  Added '+' to FOLLOW(T) from FIRST of the sequence following it.
  The sequence following T is empty or nullable.
  Propagating FOLLOW(E) to FOLLOW(T).
  Added '$' to FOLLOW(T) from FOLLOW(E).

Production: E -> T E'
Examining non-terminal 'E''.
  The sequence following E' is empty or nullable.
  Propagating FOLLOW(E) to FOLLOW(E').
  Added '$' to FOLLOW(E') from FOLLOW(E).

Production: E' -> + T E'
Examining non-terminal 'T'.
  The sequence following T is empty or nullable.
  Propagating FOLLOW(E') to FOLLOW(T).

Production: E' -> + T E'
Examining non-terminal 'E''.
  The sequence following E' is empty or nullable.
  Propagating FOLLOW(E') to FOLLOW(E').

Production: T -> F T'
Examining non-terminal 'F'.
  Added '*' to FOLLOW(F) from FIRST of the sequence following it.
  The sequence following F is empty or nullable.
  Propagating FOLLOW(T) to FOLLOW(F).
  Added '$' to FOLLOW(F) from FOLLOW(T).
  Added '+' to FOLLOW(F) from FOLLOW(T).

Production: T -> F T'
Examining non-terminal 'T''.
  The sequence following T' is empty or nullable.
  Propagating FOLLOW(T) to FOLLOW(T').
  Added '$' to FOLLOW(T') from FOLLOW(T).
  Added '+' to FOLLOW(T') from FOLLOW(T).
```

```
Production: T' -> * F T'
Examining non-terminal 'F'.
  The sequence following F is empty or nullable.
  Propagating FOLLOW(T') to FOLLOW(F).

Production: T' -> * F T'
Examining non-terminal 'T''.
  The sequence following T' is empty or nullable.
  Propagating FOLLOW(T') to FOLLOW(T').

Production: F -> ( E )
Examining non-terminal 'E'.
  Added ')' to FOLLOW(E) from FIRST of the sequence following it.

Production: E -> T E'
Examining non-terminal 'T'.
  The sequence following T is empty or nullable.
  Propagating FOLLOW(E) to FOLLOW(T).
  Added ')' to FOLLOW(T) from FOLLOW(E).

Production: E -> T E'
Examining non-terminal 'E''.
  The sequence following E' is empty or nullable.
  Propagating FOLLOW(E) to FOLLOW(E').
  Added ')' to FOLLOW(E') from FOLLOW(E).

Production: E' -> + T E'
Examining non-terminal 'T'.
  The sequence following T is empty or nullable.
  Propagating FOLLOW(E') to FOLLOW(T).

Production: E' -> + T E'
Examining non-terminal 'E''.
  The sequence following E' is empty or nullable.
  Propagating FOLLOW(E') to FOLLOW(E').

Production: T -> F T'
Examining non-terminal 'F'.
  The sequence following F is empty or nullable.
  Propagating FOLLOW(T) to FOLLOW(F).
  Added ')' to FOLLOW(F) from FOLLOW(T).
```

```
  Production: T -> F T'
  Examining non-terminal 'T''.
    The sequence following T' is empty or nullable.
    Propagating FOLLOW(T) to FOLLOW(T').
    Added ')' to FOLLOW(T') from FOLLOW(T).

  Production: T' -> * F T'
  Examining non-terminal 'F'.
    The sequence following F is empty or nullable.
    Propagating FOLLOW(T') to FOLLOW(F).

  Production: T' -> * F T'
  Examining non-terminal 'T''.
    The sequence following T' is empty or nullable.
    Propagating FOLLOW(T') to FOLLOW(T').

  Production: F -> ( E )
  Examining non-terminal 'E'.

  Production: E -> T E'
  Examining non-terminal 'T'.
    The sequence following T is empty or nullable.
    Propagating FOLLOW(E) to FOLLOW(T).

  Production: E -> T E'
  Examining non-terminal 'E''.
    The sequence following E' is empty or nullable.
    Propagating FOLLOW(E) to FOLLOW(E').

  Production: E' -> + T E'
  Examining non-terminal 'T'.
    The sequence following T is empty or nullable.
    Propagating FOLLOW(E') to FOLLOW(T).

  Production: E' -> + T E'
  Examining non-terminal 'E''.
    The sequence following E' is empty or nullable.
    Propagating FOLLOW(E') to FOLLOW(E').

  Production: T -> F T'
  Examining non-terminal 'F'.
    The sequence following F is empty or nullable.
    Propagating FOLLOW(T) to FOLLOW(F).
```

```
Production: T -> F T'
Examining non-terminal 'T''.
  The sequence following T' is empty or nullable.
  Propagating FOLLOW(T) to FOLLOW(T').

Production: T' -> * F T'
Examining non-terminal 'F'.
  The sequence following F is empty or nullable.
  Propagating FOLLOW(T') to FOLLOW(F).

Production: T' -> * F T'
Examining non-terminal 'T''.
  The sequence following T' is empty or nullable.
  Propagating FOLLOW(T') to FOLLOW(T').

Production: F -> ( E )
Examining non-terminal 'E'.

FINAL FOLLOW SETS
------------------------------------------------------------
FOLLOW(E)    = { $, ) }
FOLLOW(E')   = { $, ) }
FOLLOW(F)    = { $, ), *, + }
FOLLOW(T)    = { $, ), + }
FOLLOW(T')   = { $, ), + }

Parsing table generated successfully.
```

```
======================================================
                LL(1) PARSING TABLE
======================================================

NON-TERMINAL   $                (                )                *                +                id

-------------------------------------------------------------------------------------------------------------------------------------------------------------
----
E              -                E -> T E'        -                -                -                E -> T E'

E'             E' -> epsilon    -                E' -> epsilon    -                E' -> + T E'     -

F              -                F -> ( E )       -                -                -                F -> id

T              -                T -> F T'        -                -                -                T -> F T'

T'             T' -> epsilon    -                T' -> epsilon    T' -> * F T'     T' -> epsilon    -


Legend:
  -         : Empty table cell
  CONFLICT  : Multiple productions in the same cell
  A -> ...  : Production selected by the parser

======================================================
                CONFLICT ANALYSIS
======================================================
No parsing-table conflicts detected.
Every occupied cell contains one production.

======================================================
              GRAMMAR CLASSIFICATION
======================================================
Grammar is LL(1).
```

```
Do you want to test an input string? (y/n): y

============================================================
                      PARSING TRACE
============================================================

Enter tokens separated by spaces.
Example: id + id * id
For empty input, enter epsilon.

Input string: id + id * id
STACK                     REMAINING INPUT           ACTION
-------------------------------------------------------------------------------------
$ E                       id + id * id $            E -> T E'
$ E' T                    id + id * id $            T -> F T'
$ E' T' F                 id + id * id $            F -> id
$ E' T' id                id + id * id $            Match id
$ E' T'                   + id * id $               T' -> epsilon
$ E'                      + id * id $               E' -> + T E'
$ E' T +                  + id * id $               Match +
$ E' T                    id * id $                 T -> F T'
$ E' T' F                 id * id $                 F -> id
$ E' T' id                id * id $                 Match id
$ E' T'                   * id $                    T' -> * F T'
$ E' T' F *               * id $                    Match *
$ E' T' F                 id $                      F -> id
$ E' T' id                id $                      Match id
$ E' T'                   $                         T' -> epsilon
$ E'                      $                         E' -> epsilon
$                         $                         ACCEPT

============================================================
                      PARSING RESULT
============================================================
ACCEPTED: The input string belongs to the grammar.

============================================================
              LL(1) ANALYSIS COMPLETED
============================================================

Save the console analysis to LL1_Analysis_Report.txt? (y/n): y
Report saved as LL1 Analysis Report.txt in the current working directory.
```

```
============================================================
                  LL(1) GRAMMAR ANALYZER
============================================================
1. Analyze one grammar
2. Compare two grammars entered from keyboard
Enter your choice: 1
============================================================
                  LL(1) GRAMMAR ANALYZER
============================================================

SELECT GRAMMAR
------------------------------------------------------------
1. Arithmetic Expression Grammar
2. Simple Assignment Grammar
3. Grammar with Epsilon Productions
4. Enter Custom Grammar

Enter your choice: 4

GRAMMAR INPUT INSTRUCTIONS
------------------------------------------------------------
1. Use -> between the left and right sides.
2. Separate symbols using spaces.
3. Use | for alternatives.
4. Use epsilon for an empty production.
5. Do not use $ as a grammar symbol.

Enter the number of production lines: 3

Enter your grammar:
Production 1: S -> a A | a B
Production 2: A -> b
Production 3: B -> c

Grammar entered successfully.

============================================================
                     ENTERED GRAMMAR
============================================================
  1. S -> a A
  2. S -> a B
  3. A -> b
  4. B -> c
```

```
Start symbol : S
Non-terminals: A B S
Terminals    : a b c

============================================================
               DIRECT LEFT-RECURSION CHECK
============================================================
No direct left recursion detected.
Note: this check detects direct left recursion; it does not prove that indirect left recursion is absent.

============================================================
             GRAMMAR RELATIONSHIP VIEW
============================================================
Start symbol: S

Production paths (non-terminals point to symbols on their RHS):
[A]
  +-- 'b'
[B]
  +-- 'c'
[S]
  +-- 'a' [A]
  +-- 'a' [B]

Legend: [Name] = non-terminal; 'token' = terminal.

Show step-by-step FIRST and FOLLOW calculations? (y/n): n

============================================================
                    FIRST SET CALCULATION
============================================================

FINAL FIRST SETS
------------------------------------------------------------
FIRST(A)    = { b }
FIRST(B)    = { c }
FIRST(S)    = { a }
```

```
============================================================
                  FOLLOW SET CALCULATION
============================================================

FINAL FOLLOW SETS
------------------------------------------------------------
FOLLOW(A)   = { $ }
FOLLOW(B)   = { $ }
FOLLOW(S)   = { $ }

Parsing table generated successfully.

============================================================
                  LL(1) PARSING TABLE
============================================================

NON-TERMINAL   $                   a                   b                   c
-------------------------------------------------------------------------------------------------------
A              -                   -                   A -> b              -
B              -                   -                   -                   B -> c
S              -                   CONFLICT            -                   -

Legend:
  -         : Empty table cell
  CONFLICT  : Multiple productions in the same cell
  A -> ...  : Production selected by the parser

============================================================
                  CONFLICT ANALYSIS
============================================================

Conflict at M[S, a]
Conflicting productions:
  1. S -> a A
  2. S -> a B

Explanation:
The lookahead symbol 'a' selects more than one production for S.
A predictive parser therefore cannot choose one production uniquely.

Possible improvement:
The conflicting alternatives begin with 'a'. Consider left factoring the grammar, then analyze the transformed grammar again.
Conclusion: The grammar is NOT LL(1).
```

```
------------------------------------------------------------


============================================================
                  GRAMMAR CLASSIFICATION
============================================================
Grammar is NOT LL(1).
Predictive parsing is skipped because the table has conflicts.

Would you like to generate and analyze a left-factored version? (y/n): y

============================================================
             AUTOMATIC LEFT-FACTORING RESULT
============================================================
Generated grammar (review it before using it in a formal submission):
S -> a S'
A -> b
B -> c
S' -> A
S' -> B

The analyzer will now recompute FIRST, FOLLOW, and the parsing table.

============================================================
                      ENTERED GRAMMAR
============================================================
1  . S -> a S'
2  . A -> b
3  . B -> c
4  . S' -> A
5  . S' -> B

Start symbol : S
Non-terminals: A B S S'
Terminals    : a b c
```

```
============================================================
                  FIRST SET CALCULATION
============================================================

FINAL FIRST SETS
------------------------------------------------------------
FIRST(A)    = { b }
FIRST(B)    = { c }
FIRST(S)    = { a }
FIRST(S')   = { b, c }

============================================================
                  FOLLOW SET CALCULATION
============================================================

FINAL FOLLOW SETS
------------------------------------------------------------
FOLLOW(A)   = { $ }
FOLLOW(B)   = { $ }
FOLLOW(S)   = { $ }
FOLLOW(S')  = { $ }

Parsing table generated successfully.

============================================================
                   LL(1) PARSING TABLE
============================================================

NON-TERMINAL   $                   a                   b                   c
-------------------------------------------------------------------------------------------------------
A              -                   -                   A -> b              -
B              -                   -                   -                   B -> c
S              -                   S -> a S'           -                   -
S'             -                   -                   S' -> A             S' -> B

Legend:
  -          : Empty table cell
  CONFLICT   : Multiple productions in the same cell
  A -> ...   : Production selected by the parser
```

```
============================================================
                    CONFLICT ANALYSIS
============================================================
No parsing-table conflicts detected.
Every occupied cell contains one production.

Classification after left factoring: LL(1)

Do you want to test an input string with the transformed grammar? (y/n): y

============================================================
                      PARSING TRACE
============================================================

Enter tokens separated by spaces.
Example: id + id * id
For empty input, enter epsilon.

Input string: id + id

PARSING ERROR
Unexpected token: 'id'
Token position: 1
Reason: this token does not appear among the grammar's terminals.
Action: check the token spelling or revise the grammar.
Result: REJECTED
```